

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Криптографические методы обеспечения информационной безопасности»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Асимметричные криптосистемы»

Выполнили:

Лим Алина Олеговна, студент группы N3349



Проверил:

Таранов Сергей Владимирович, доцент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение	3
1 Ход работы	4
1.1 Шифрование.....	4
1.2 Атака на шифр	6
1.3 Программная реализация алгоритма	6
1.4 Модификация решения	8
Заключение.....	10

ВВЕДЕНИЕ

Цель работы: изучить основные принципы работы асимметричных криптосистем на примере алгоритма RSA.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать эмуляцию алгоритма RSA и примитивных атак на шифр, используя Cryptool 2. Выделить основные необходимые настройки шифра и требуемые ограничения на параметры;
- программно реализовать и модифицировать любую асимметричную криптосистему. В случае отсутствия опыта программирования подойдет реализация алгоритма на псевдокоде или в виде блок схем, включающих основные этапы алгоритма с отображением формул и основных математических действий. Атаки и модификации, приведенные ниже, указаны для RSA. Если атака или модификация не применима для реализуемого алгоритма разрешается найти любую альтернативу (атаки, применимой к алгоритму; модификации для ускорения алгоритма и дополнительной защиты);
- для созданной реализации криптосистемы предлагается провести примитивный криптоанализ на устойчивость к следующим атакам, а также сделать минимальные модификации по оптимизации (ускорению процессов шифрования, дешифрования, процесса генерации ключей).

1 ХОД РАБОТЫ

1.1 Шифрование

Зашифруем текст с помощью RSA.

Текст на входе: «Georges Méliès, one of the "pioneers of cinema", the discoverer of the trick possibilities of cinema and the founder of its "formative" or "poetic" line, made the trick an integral element of his artistic style, thanks to which the art of cinema for the first time turned to a tradition based on the creative transformation of reality. Having become the inventor of special effects editing, Melies laid the foundation for the development of a number of film genres based on the spectacular principle. Many of the tricks and techniques he invented are still used in cinema, albeit in an improved form, to create the most modern types of special effects».

Для начала зашифруем текст со случайно сгенерированными параметрами.

Результат представлен на рисунке 1.

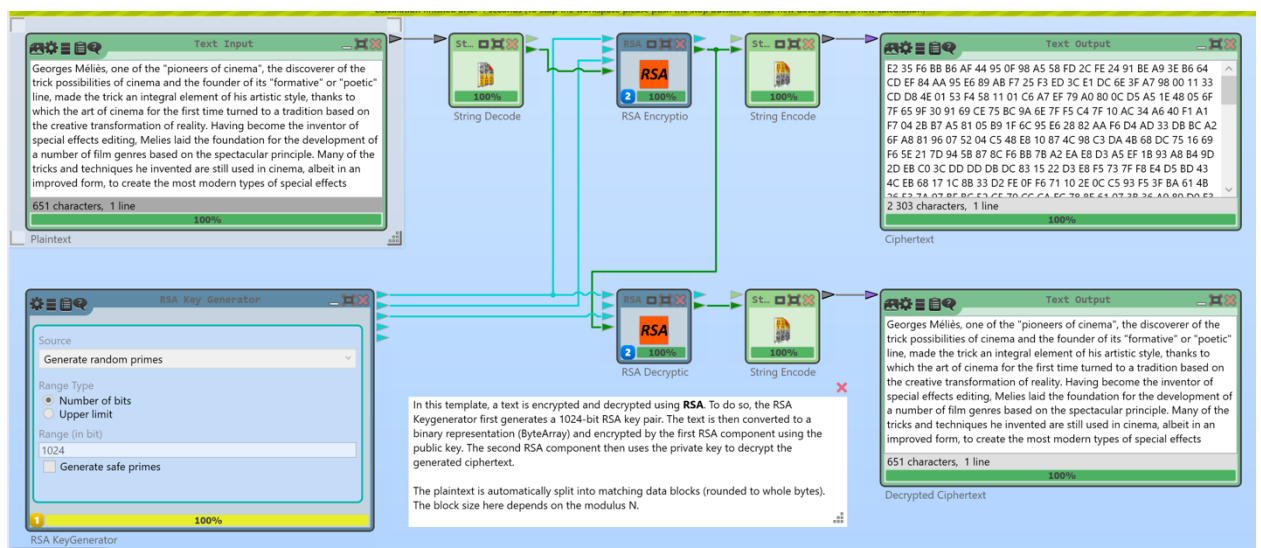


Рисунок 1 – Шифрование со случайными параметрами

Далее зашифруем этот же текст с указанными параметрами:

$N = 3233$;

$e = 65537$;

$d = 2753$.

Результат представлен на рисунке 2.

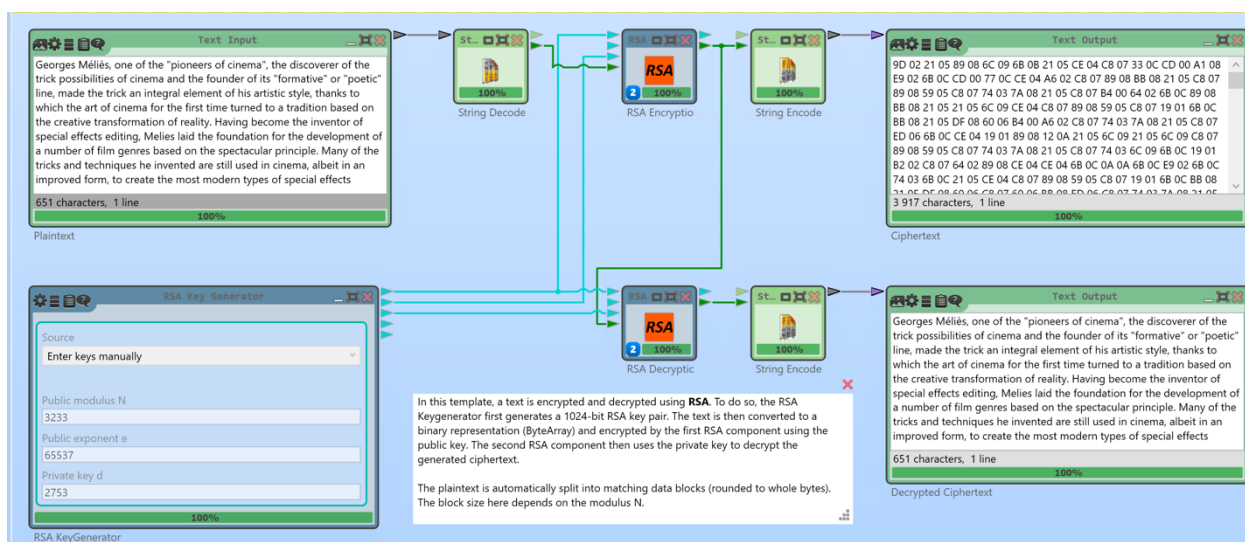


Рисунок 2 – Шифрование текста с заданными параметрами

Для параметров применяются следующие ограничения:

- для простых чисел p и q :
 - разность $|p-q|$ должна быть большой;
 - рекомендуется использовать простые числа с длиной битов, обычно равной 2048 битам или более, для обеспечения достаточной стойкости к взлому;
 - более сильные уровни безопасности могут требовать ключей 3072 бита и выше;
 - числа $p \pm 1$, $q \pm 1$ должны содержать большой простой множитель;
 - $\text{НОД}(p-1, q-1)$ должен быть небольшим.
- для параметров N и $\phi(N)$:
 - $N=p \cdot q$, и его размер равен произведению размеров p и q ;
 - простое число N должно быть большим;
 - $\phi(N)=(p-1) \cdot (q-1)$ и также имеет соответствующий размер.
- для открытый ключа e и закрытого ключа d :
 - открытый экспонент e обычно выбирается как небольшое простое число;
 - вычисление d обычно выполняется с использованием расширенного алгоритма Евклида или других алгоритмов, обеспечивая долгое и безопасное значение;
 - не стоит выбирать ключи e и d с короткой секретной экспонентой;

- открытый ключ (содержащий e и N) распространяется для шифрования и верификации подписей.

1.2 Атака на шифр

Реализуем атаку на шифр *RSA Common Factor Attack*, зашифровав два текста помощью различных модулей шифрования N , соответствующих условиям, перечисленным в разделе 1.1, с общим делителем.

Результат представлен на рисунке 3.

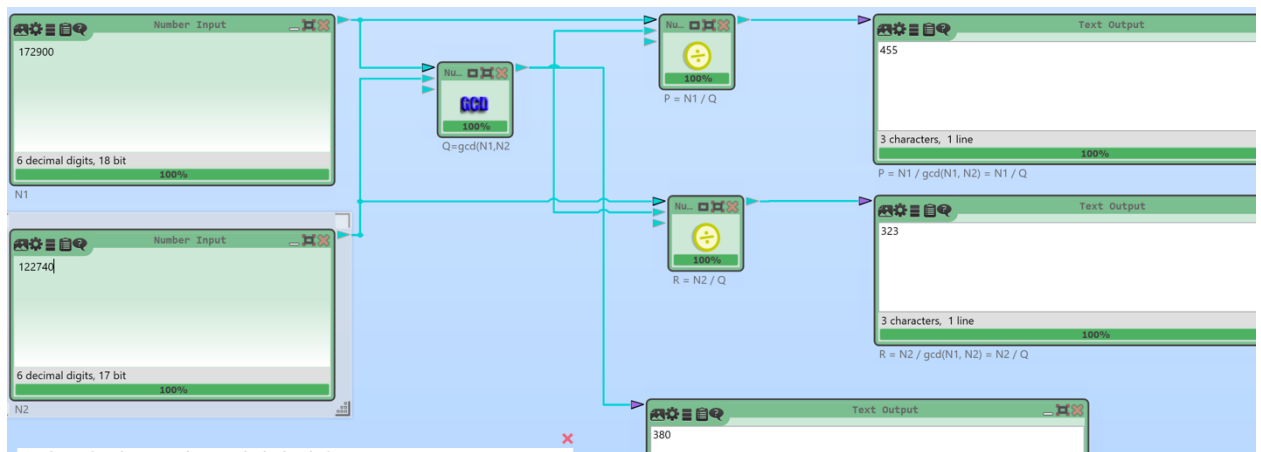


Рисунок 3 – Атака на шифр

1.3 Программная реализация алгоритма

Реализуем алгоритм RSA на языке C++ (листинг 1, 2).

Листинг 1 – Клиентская часть

```
# include <bits/stdc++.h>
# include <arpa/inet.h>
using namespace std;
int connectToServer(const char* ip, int port)
{
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {AF_INET, htons(port), inet_addr(ip)};
    if(connect(sock, (struct sockaddr *) &addr, sizeof(addr)) < 0 )
    {
        cout << "\nRun server program first." << endl;
        exit(0);
    }
    else
    {
        cout << "\nClient is connected to Server." << endl;
    }
    return sock;
}
int randInRange(int low, int high) // excluding high and low
{
    return rand()%(high-(low+1)) + (low+1) ;
}
```

```

}
int gcd(int a, int b)
{
    return b==0 ? a : gcd(b, a%b);
}
int powermod(int a, int b, int n)
{
    int res = 1;
    for(int i=0; i<b; i++)
    {
        res = (res*a) % n;
    }
    return res;
}
//  $M = C^d \bmod n$ 
int decrypt(int C, int PR[2])
{
    return powermod(C, PR[0], PR[1]);
}
int main()
{
    char ip[50];
    int port;
    cout << "\nEnter server's IP address: "; cin >> ip;
    cout << "Enter port : "; cin >> port;
    int sock = connectToServer(ip, port);
    int p,q;
    cout << "\nEnter two prime numbers : "; cin >> p >> q;
    int n = p * q ;
    int phi = (p-1) * (q-1);
    srand(time(NULL));
    int e, d;
    do
    {
        e = randInRange(1, phi);
    }
    while(gcd(e,phi) != 1);
    for(d=1; d<phi; d++)
    {
        if((d*e)%phi == 1)
            break;
    }
    int PU[2] = {e, n}; // public key
    int PR[2] = {d, n}; // private key
    cout << "\nPublic key , PU = {" << e << ", " << n << "}" << endl;
    cout << "Private key, PR = {" << d << ", " << n << "}" << endl;
    send(sock, &PU, sizeof(PU), 0); // send public key to server
    cout << "\nSent Public key to server." << endl;
    int C; // ciphertext
    recv(sock, &C, sizeof(C), 0); // receive ciphertext from server
    cout << "\nCiphertext received from server : " << C << endl;
    int M = decrypt(C, PR); // decrypted text
    cout << "\nDecrypted Text : " << M << endl << endl;
}

```

Листинг 2 – Серверная часть

```

# include <bits/stdc++.h>
# include <arpa/inet.h>
using namespace std;
int createServer(int port) // TCP connection
{

```

```

    int sersock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {AF_INET, htons(port), INADDR_ANY};
    bind(sersock, (struct sockaddr *) &addr, sizeof(addr));
    cout << "\nServer Online. Waiting for client..." << endl;
    listen(sersock, 5);
    int sock = accept(sersock, NULL, NULL);
    cout << "Connection Established." << endl;
    return sock;
}
int powermod(int a, int b, int n)
{
    int res = 1;
    for(int i=0; i<b; i++)
    {
        res = (res*a) % n;
    }
    return res;
}
//  $C = M^e \bmod n$ 
int encrypt(int M, int PU[2]) // PU = {e, n}
{
    return powermod(M, PU[0], PU[1]);
}
int main()
{
    int port;
    cout << "\nEnter port : "; cin >> port;
    int sock = createServer(port);
    int PU[2];
    recv(sock, &PU, sizeof(PU), 0); // receive public key from client
    cout << "\nPublic key received from client : {" << PU[0] << ", " <<
    PU[1] << "}" << endl;
    int M; // plaintext message (M < n)
    cout << "\nEnter message(M<" << PU[1] << ") to encrypt : "; cin >> M;
    int C = encrypt(M, PU);
    cout << "\nEncrypted Text : " << C << endl;
    send(sock, &C, sizeof(C), 0); // send ciphertext to client
    cout << "\nSent ciphertext to client." << endl << endl;
}

```

1.4 Модификация решения

Рассмотрим возможные улучшения: выбор достаточно большой экспоненты (листинг 3), алгоритм быстрого возведения в степень (листинг 4), китайская теорема об остатках (листинг 5).

Листинг 3 – Выбор достаточно большой экспоненты

```

//RSA
do
{
    // e = FermiNumbers[rand() % 5];
    e = 100000 + rand() % 900000;
}
while (gcd(e, euler) != 1 || e >= euler);
//RSA

```

Листинг 4 – Быстрое возведение в степень

```

powMod(a, b, n)

```

```

{
    x = 1;
    a %= n;
    while (b > 0)
    {
        if (b & 1)
        {
            x = mulmod(x, a, n);
        }
        a = mulmod(a, a, n);
        b >>= 1;
    }
    return x % n;
}

```

Листинг 5 – Китайская теорема об остатках

```

for (int i = 0; i < message.size(); i++)
{
    m1 = powMod(encryptedMessage[i], dP, PrimeNumbers[0]);
    m2 = powMod(encryptedMessage[i], dQ, PrimeNumbers[1]);
    h = qInv * (m1 - m2) % PrimeNumbers[0];
    m = (m2 + h * PrimeNumbers[1]) % modul;
    std::cout << (char)m;
}

```

ЗАКЛЮЧЕНИЕ

В ходе работы была проанализирована эмуляция алгоритма RSA и примитивная атака на шифр (RSA Common Factor Attack), с использованием Cryptool 2. Были выделены основные необходимые настройки шифра и требуемые ограничения на параметры. Также была программно реализована и модифицирована асимметричная криптосистема RSA. Это помогло в изучении основных принципов работы асимметричных криптосистем на примере алгоритма RSA. Таким образом, поставленные задачи были выполнены, цель достигнута.